TASK 10: Write HBase queries to handle databases.

1.Start HBase: hbase shell

2.Create Tables with Multiple Columns:

create 'students', 'personal', 'academic', 'contact'

```
hbase(main):035:0>
hbase(main):036:0* create 'students', 'personal', 'academic', 'contact'
0 row(s) in 2.3480 seconds

=> Hbase::Table - students
hbase(main):037:0>
hbase(main):038:0*
```

3.Insert Data Into HBase Table:

put 'students', '1', 'personal:name', 'Alice Johnson'
put 'students', '1', 'personal:age', '22'
put 'students', '1', 'academic:department', 'Computer Science'
put 'students', '1', 'academic:marks', '89.5'
put 'students', '1', 'contact:email', 'alice@example.com'
put 'students', '1', 'contact:phone', '9876543210'

put 'students', '2', 'personal:name', 'Bob Smith'
put 'students', '2', 'personal:age', '21'
put 'students', '2', 'academic:department', 'Mathematics'
put 'students', '2', 'academic:marks', '76.0'
put 'students', '2', 'contact:email', 'bob@example.com'
put 'students', '2', 'contact:phone', '8765432109'

```
hbase(main):038:0* put 'students', '1', 'personal:name', 'Alice Johnson'
dents', '1', 'personal:age', 0 row(s) in 0.0290 seconds

'22'

put 'students', '1', 'academic:hbase(main):039:0>
hbase(main):040:0* put 'students', '1', 'personal:age', '22'
department', 'Computer Science'

put 'students', '1', 0 row(s) in 0.0080 seconds

'academic:marks', '89.5hbase(main):041:0>
hbase(main):042:0* put 'students', '1', 'academic:department', 'Computer Scienc
'

put 'students', '1', 'contact:e0 row(s) in 0.0110 seconds

hbase(main):043:0>
mail', 'alice@example.com'hbase(main):044:0* put 'students', '1', 'academic:mar
s', '89.5'
```

```
hbase(main):059:0>
hbase(main):060:0* put 'students', '2', 'contact:email', 'bob@example.com'
0 row(s) in 0.0070 seconds

hbase(main):061:0>
hbase(main):062:0* put 'students', '2', 'contact:phone', '8765432109'
0 row(s) in 0.0070 seconds

hbase(main):063:0>
hbase(main):064:0*
```

4.Retrieve Data from HBase Table

a. Fetch all records: scan 'students'

```
hbase(main):069:0* scan 'students'
ROW                    COLUMN+CELL

ERROR: Connection refused

Scan a table; pass table name and optionally a dictionary of scanner
specifications.  Scanner specifications may include one or more of:
TIMERANGE, FILTER, LIMIT, STARTROW, STOPROW, ROWPREFIXFILTER, TIMESTAMP,
MAXLENGTH or COLUMNS, CACHE or RAW, VERSIONS, ALL_METRICS or METRICS

If no columns are specified, all columns will be scanned.
To scan all members of a column family, leave the qualifier empty as in
'col_family'.

The filter can be specified in two ways:
1. Using a filterString - more information on this is available in the
Filter Language document attached to the HBASE-4176 JIRA
2. Using the entire package name of the filter.

If you wish to see metrics regarding the execution of the scan, the
ALL_METRICS boolean should be set to true. Alternatively, if you would
prefer to see only a subset of the metrics, the METRICS array can be
defined to include the names of only the metrics you care about.
```

```
ring)
 2. or as a custom class followed by method name: e.g. 'c(MyFormatterClass).fo
at'.

Example formatting cf:qualifier1 and cf:qualifier2 both as Integers:
  hbase> scan 't1', {COLUMNS => ['cf:qualifier1:toInt',
    'cf:qualifier2:c(org.apache.hadoop.hbase.util.Bytes).toInt'] }

Note that you can specify a FORMATTER by column only (cf:qualifier).  You cann
specify a FORMATTER for all columns of a column family.

Scan can also be used directly from a table, by first getting a reference to a
table, like such:

  hbase> t = get_table 't'
  hbase> t.scan

Note in the above situation, you can still provide all the filtering, columns,
options, etc as described above.


hbase(main):070:0>
hbase(main):071:0*
```

b. Fetch a specific row: get 'students', '1'

```
hbase(main):005:0>
hbase(main):006:0* get 'students', '1'
COLUMN                 CELL
 academic:department   timestamp=1742552705484, value=Computer Science
 academic:marks        timestamp=1742552705527, value=89.5
 contact:email         timestamp=1742552705565, value=alice@example.com
 contact:phone         timestamp=1742552705671, value=9876543210
 personal:age          timestamp=1742552705443, value=22
 personal:name         timestamp=1742552705374, value=Alice Johnson
6 row(s) in 0.1100 seconds

hbase(main):007:0>
hbase(main):008:0*
```

c. Fetch a specific column value: get 'students', '2', 'personal:name'

```
hbase(main):007:0>
hbase(main):008:0* get 'students', '2', 'personal:name'
COLUMN                 CELL
 personal:name         timestamp=1742552705749, value=Bob Smith
1 row(s) in 0.0230 seconds

hbase(main):009:0>
hbase(main):010:0*
```

d. Fetch multiple columns in a row: get 'students', '1', {COLUMNS => ['personal:name', 'academic:marks']}

```
hbase(main):009:0>
hbase(main):010:0* get 'students', '1', {COLUMNS => ['personal:name', 'academic:
marks']}
COLUMN                 CELL
 academic:marks        timestamp=1742552705527, value=89.5
 personal:name         timestamp=1742552705374, value=Alice Johnson
2 row(s) in 0.0270 seconds

hbase(main):011:0>
hbase(main):012:0*
```

5. **Update Data in HBase:** put 'students', '1', 'academic:marks', '95.0'

```
hbase(main):011:0>
hbase(main):012:0* put 'students', '1', 'academic:marks', '95.0'
0 row(s) in 0.1180 seconds

hbase(main):013:0>
hbase(main):014:0*
```

6. **Delete Data:** **delete 'students', '2', 'contact:email'**

```
hbase(main):015:0>
hbase(main):016:0* delete 'students', '2', 'contact:email'
0 row(s) in 0.0650 seconds
```

7. **Create Another Table:** **create 'courses', 'info', 'professor'**

```
hbase(main):022:0>
hbase(main):023:0* create 'courses', 'info', 'professor'
0 row(s) in 2.5950 seconds
```

Insert records:

put 'courses', 'CS101', 'info:name', 'Data Structures'
put 'courses', 'CS101', 'info:credits', '4'
put 'courses', 'CS101', 'professor:name', 'Dr. Lee'

put 'courses', 'MATH201', 'info:name', 'Linear Algebra'
put 'courses', 'MATH201', 'info:credits', '3'
put 'courses', 'MATH201', 'professor:name', 'Dr. Smith'

```
hbase(main):043:0> put 'courses', 'CS101', 'info:credits', '4'
', 'CS101', 'prof0 row(s) in 0.0170 seconds

essor:name', 'Dr. Lee'hbase(main):044:0>
hbase(main):045:0* put 'courses', 'CS101', 'professor:name', 'Dr. Lee'
0 row(s) in 0.0250 seconds

hbase(main):046:0> put 'courses', 'MATH201', 'info:name', 'Linear Algebra'
0 row(s) in 0.0240 seconds

hbase(main):047:0> put 'courses', 'MATH201', 'info:credits', '3'
0 row(s) in 0.0280 seconds

hbase(main):048:0> put 'courses', 'MATH201', 'professor:name', 'Dr. Smith'
0 row(s) in 0.0250 seconds

hbase(main):049:0> get 'courses', 'CS101'
COLUMN                    CELL
 info:credits             timestamp=1742638991938, value=4
```

Retrieve course details: **get 'courses', 'CS101'**

```
hbase(main):049:0> get 'courses', 'CS101'
COLUMN                    CELL
 info:credits             timestamp=1742638991938, value=4
```

8. **List and Count Tables:**
   a. **List all tables:** **list**

```
hbase(main):050:0>
hbase(main):051:0* list
TABLE
courses
students
2 row(s) in 0.0440 seconds

=> ["courses", "students"]
hbase(main):052:0>
hbase(main):053:0*
```

   b. **Count total records in a table: count 'students'**
      count 'courses'

```
hbase(main):052:0>
hbase(main):053:0* count 'students'
1 row(s) in 0.0330 seconds

=> 1
hbase(main):054:0>
hbase(main):055:0* count courses
NameError: undefined local variable or method `courses' for #<Object:0x5324123>

hbase(main):056:0> count 'courses'
2 row(s) in 0.0280 seconds

=> 2
hbase(main):057:0>
```

9. **Using Filters in HBase:**
   a. **Retrieve students from the Computer Science department:**
      scan 'students', {FILTER => "SingleColumnValueFilter('academic', 'department', =, 'binary:Computer Science')"}

```
hbase(main):057:0> scan 'students', {FILTER => "SingleColumnValueFilter('academi
c', 'department', =, 'binary:Computer Science')"}
ROW                    COLUMN+CELL
 1                     column=academic:department, timestamp=1742552705484, value
                       =Computer Science
 1                     column=academic:marks, timestamp=1742636011185, value=95.0
 1                     column=contact:phone, timestamp=1742552705671, value=98765
                       43210
 1                     column=personal:name, timestamp=1742552705374, value=Alice
                        Johnson
1 row(s) in 0.1250 seconds

hbase(main):058:0>
hbase(main):059:0*
```

b. **Retrieve students who scored more than 80 marks**

scan 'students', {FILTER => "SingleColumnValueFilter('academic', 'marks', >, 'binary:80')"}

```
hbase(main):058:0>
hbase(main):059:0* scan 'students', {FILTER => "SingleColumnValueFilter('academi
c', 'marks', >, 'binary:80')"}
ROW                    COLUMN+CELL
 1                     column=academic:department, timestamp=1742552705484, value
                       =Computer Science
 1                     column=academic:marks, timestamp=1742636011185, value=95.0
 1                     column=contact:phone, timestamp=1742552705671, value=98765
                       43210
 1                     column=personal:name, timestamp=1742552705374, value=Alice
                        Johnson
1 row(s) in 0.0510 seconds

hbase(main):060:0>
hbase(main):061:0*
```

10. Disable and Drop Tables:

disable 'students'
drop 'students'

disable 'courses'
drop 'courses'

```
hbase(main):060:0>
hbase(main):061:0* disable 'students'
0 row(s) in 2.5790 seconds

hbase(main):062:0> drop 'students'
0 row(s) in 2.4770 seconds

hbase(main):063:0> disable 'courses'
0 row(s) in 2.4440 seconds

hbase(main):064:0> drop 'courses'
0 row(s) in 2.4260 seconds

hbase(main):065:0>
```